全球人工智能算法精英赛 · AI + 硬件赛道

RSI-EdgeVision

端侧遥感变化检测系统

EDGE-SIDE REMOTE SENSING CHANGE DETECTION ON RDK X5

3.2 MB	4 GB	0.8648	int8
量化模型体积	板端内存约束	LEVIR-CD F1	BPU 定点推理

参赛参考资料 V1.0

2026 年 9 月

作品速览

一句话读懂这个作品

一句话定义：把「双时相遥感变化检测」这套像素级 AI 算法，从 GPU 服务器完整搬进一块 **4 GB 内存、无独立显卡** 的国产边缘算力板卡（RDK X5），并在板端跑通检测、评估、误差分析与语义解释的完整闭环。

关键指标

0.8782	0.8517	0.8648	0.7617
PRECISION	RECALL	F1	IoU
3.2 MB	**60 组**	**256×256**	**6 通道**
模型体积（原 10.2 MB）	真实校准样本	单帧输入分辨率	双时相拼接输入

为什么这个作品属于「AI + 硬件」

算法侧	自研轻量化变化检测网络 **REPB-Net**，以共享 MobileNetV2 为骨干，配合 RTRF、MSRB、SAM 三个自研模块，在压低参数量的同时守住变化边界的细节
硬件侧	面向 RDK X5 的 **bayes-e BPU** 架构，以 int8 定点运算为主、仅支持有限算子集，且板卡内存只有 4 GB
协同侧	打通 **PyTorch → ONNX → 真实数据校准量化 → BPU** 全链路，模型压缩至 3.2 MB，量化后精度与浮点模型基本持平

赛道契合点：本作品不是「把模型拷到板子上试试」，而是围绕硬件的算力、内存与算子约束， 对算法结构、导出方式、量化策略做了**系统性适配**——这正是 AI + 硬件赛道最核心的考察方向。

背景与意义

AI 能力与算力供给之间的鸿沟，正是本作品的出发点

一、产业背景：模型越来越大，算力却到不了现场

近年来视觉深度网络在精度上持续突破，但参数量与算力需求同步膨胀，推理环节高度依赖云端 GPU 集群。 在灾害应急、野外巡检、工业现场等真实场景中，云端推理面临四重约束：

时延	带宽	合规	可用性
上行 + 排队不可控，灾后按小时计不可接受	单景影像百兆级，回传成本高	遥感数据出域存在安全风险	野外断网即失效

> **结论：**把 AI 算法真正放到设备端，已成为智能感知系统落地的必答题，也是国产边缘算力的核心命题。

二、场景选择：为什么是遥感变化检测

遥感变化检测要求从同一区域不同时相的影像中识别地物变化，是灾后损毁评估、违建与施工监管、 国土动态监测的基础能力。该任务对端侧智能提出了典型而严苛的三重要求：

- **时效以小时计**——灾害应急场景下，等待回传与排队不可接受；
- **数据量大**——单景遥感影像达百兆级，本地处理远优于上传；
- **现场无网**——无人机、巡检车作业区域普遍没有稳定网络。

换言之，这不是一个"顺手找来的"应用场景，而是端侧智能**最刚需、也最苛刻**的验证场。

三、双层挑战：算法要轻，硬件又要能跑

层面	具体约束
算法层	双时相影像存在光照、阴影、季节等**非真实变化干扰**；变化目标**尺度跨度大**；**边界细节易丢失**，直接影响面积统计。直接裁剪轻量网络会造成精度显著下降。
硬件层	边缘芯片以 **int8 定点运算**为主，仅支持有限算子集；板卡**内存仅 4 GB、无独立 GPU**，浮点模型无法直接运行。
叠加效应	算法要保精度就需要更多参数与更高数值精度，硬件却同时限制着体积、带宽与数值格式——**精度、体积、速度三者难以同时满足**，这正是本作品要解决的核心矛盾。

四、现有方案的不足

- **学术侧：**研究多在 GPU 服务器上完成精度验证，**缺少部署环节**，模型能否落板无人回答；
- **端侧侧：**现有边缘智能工作集中于图像分类与目标检测，**像素级语义分割的量化部署实践仍然稀缺**；
- **云侧：**云端方案精度虽高，却无法满足野外离线作业，也回避了数据出域的合规问题。

五、本项目的意义

技术意义	工程意义	产业意义
协同设计范式	**4 GB 跑通闭环**	**数据不出域**
算法结构与量化策略联合优化，形成可复制路径	在国产边缘平台实现检测·评估·解释一体化	离线部署于无人机与巡检车，服务灾损评估与国土监测

六大核心创新

算法三项 + 端侧部署三项，逐条对应前面的挑战

对应挑战	创新点	解决了什么
光照阴影伪变化	**创新 ①** **RTRF** 互易时相响应融合	不只看"差异"，而是同时构建**差异、能量、一致性**三类时相证据。一致性响应负责识别稳定背景，使模型能判断"这个差异是否有真实变化证据"，从源头压制阴影与光谱偏移造成的误检。
尺度跨度大	**创新 ②** **MSRB** 多尺度细化桥	在解码前做多尺度桥接：SCA 分支提取跨尺度共享语义，让建筑群、道路、裸地等不同尺度变化保持语义一致；CDF 分支把细粒度边界与粗粒度语义结合，补回被下采样丢失的轮廓。
边界细节易丢失	**创新 ③** **SAM** 层级注意力解码器	在多尺度输出侧预测图，用监督注意力模块把前景/背景概率反向作用于特征表达，解码过程逐级聚焦真实变化区域；训练用多输出 BCE-Dice 损失增强收敛稳定性。
硬件只吃 int8	**创新 ④** **真实校准量化**	摒弃"随机数据凑合校准"，从训练集抽取 **60 组真实 A/B 影像**做与训练一致的预处理，生成 float32 裸数据校准集，让量化器按真实激活分布定标，量化后精度基本无损。
预处理链路易断	**创新 ⑤** **四端预处理一致性**	浮点自检、ONNX 导出、校准数据生成、板端推理**四处使用同一套预处理**（BGR→拼 6 通道→归一化→减均值除标准差→通道重排），任一环不一致结果即崩，链路全程可复现。
内存仅 4 GB	**创新 ⑥** **端侧一体化工作站**	在板端实现"检测 → 指标评估 → 误差分析 → AI 语义解释 → 结果导出"的完整闭环，配合设备温度/内存/CPU 实时监控，证明受限硬件也能承载完整的工程级应用。

REPB-Net 网络架构

共享编码器 + 三个自研模块，从双时相影像到像素级变化概率图



图 1　REPB-Net 整体架构：双时相影像经共享编码器进入同一语义空间，再经 RTRF、MSRB、SAM 三级处理输出变化概率图

关键设计取舍

为什么用共享编码器	两个时相走同一套权重，特征落在同一语义空间，比较才有意义；同时参数量减半，为端侧部署留出余量。
为什么不用 Transformer	Transformer / Mamba 精度强，但参数量与访存开销大，难以在 4 GB、无 GPU 的板卡上落地。MobileNetV2 的倒残差与深度可分离卷积在低参数量下仍能提供稳定多尺度特征，是精度与可部署性的平衡点。
为什么不用简单差分	FC-Conc / FC-Diff 只"看差异"，亮度偏移与阴影会在差分空间产生强响应。RTRF 引入一致性证据，让模型能区分"真变化"与"看起来变了"。
为什么在解码前加桥	变化既可能是建筑群（区域级），也可能是道路边线（细长级）。MSRB 在解码前统一多尺度语义，避免单一解码器顾此失彼。

量化友好性：全卷积结构 + 共享权重 + 无动态控制流，使网络在转成 int8 后不会出现算子不支持或数值坍塌问题——**算法设计阶段就为量化做了预留**，这是能一次性跑通量化链路的前提。

从浮点权重到 BPU 定点模型

六个环节，每一步都对应一次真实的工程决策



图 2　端侧部署全链路：PyTorch → ONNX → 真实校准量化 → RDK X5 BPU

关键参数速查

量化架构	`march: bayes-e` ——RDK X5 的 BPU 架构，换板卡必须同步修改
输入规格	`input_name: "x1; x2"`，`input_shape: "1x3x256x256; 1x3x256x256"`，双输入参数一律用**分号**分隔写同样多份
量化方式	`calibration_type: default` + `preprocess_on: False` ——用真实数据校准，且预处理已在生成校准集时完成
编译策略	`compile_mode: latency`，`optimize_level: O2`，`core_num: 1` ——面向低延迟单帧推理

最容易踩的坑：多输入模型的参数必须写成同样多份（如 `cal_data_type: "float32; float32"`）。只写一份会直接报 `Wrong cal_data_type num given`，工具在参数数量上不做默认补齐。

一次真实的量化故障排查

这段过程最能体现"算法与硬件协同"的工程判断力，建议在答辩中重点讲

现象

第一次量化采用 `calibration_type: skip`（不提供真实数据，由工具用随机数据校准）。 工具链报告 `Convert to runtime bin file successfully`，产物大小 2,820,223 字节，看起来一切正常。 但拷到板子上跑真实影像，输出的变化像素占比是 **0.0000**——模型"瞎了"。

根因

量化需要知道每一层激活值的真实分布范围，才能把 float32 映射到 int8。 用随机数据算出的分布完全不对，量化后数值坍塌，输出几乎恒定为一个常数。 **skip 只适合验证模型结构能否通过检查，绝不能用于真实推理。**

定位方法：先"分锅"，再动手

第 1 步	第 2 步	第 3 步
写浮点自检脚本	**看浮点结果**	**得出结论**
绕过量化，直接用 .pth 在 CPU 上跑同一张图，看模型本身能否检出变化	预测变化占比 0.1885，真实标签 0.2012 —— 几乎一致	模型与预处理完全正确，问题 100% 出在 skip 量化

为什么这一步关键：如果不做浮点自检，很容易误判成"模型不行 / 预处理写错了"， 进而在错误的方向上反复重训。先用浮点结果把责任划分清楚，才能精准定位到量化环节。

修复与验证

步骤	做法与结果
生成真实校准集	从训练集抽 60 组 A/B，按训练同款预处理生成 x1 / x2 两套 float32 裸数据，每组 786,432 字节
改用 default 校准	`calibration_type: default`， `preprocess_on: False`，校准目录用分号分隔双输入
重新量化	产物由 2,820,223 字节变为 **3,279,032 字节**——模型不再坍塌，权重信息更有效
板端复测	全量测试集 F1 达到 **0.8648**，量化后精度与浮点模型基本持平

可复用的经验：①量化必须用真实数据校准；②精度异常时先在浮点侧验证，别急着重训； ③浮点自检、ONNX 导出、校准数据生成、板端推理**四处预处理必须完全一致**，任一处不一致结果都会崩。

实验结果

全部指标在 RDK X5 板端以 int8 量化模型实测

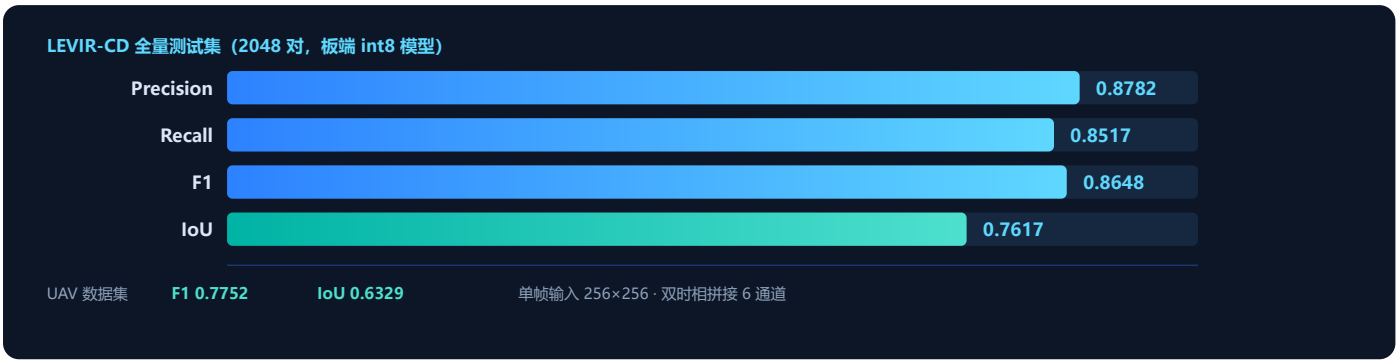


图 3　板端量化模型精度指标

量化前后精度对比

对比项	浮点模型（CPU）	板端 int8 模型	结论
变化像素占比	0.1885	—	浮点自检的预测占比（0.1885）与真实标签占比（0.2012）高度接近，说明**模型与预处理正确**；量化后板端 F1 仍达 0.8648，说明**量化基本无损。**
真实标签占比	0.2012	—	
F1	—	**0.8648**	端侧精度对标主流方法
模型体积	10.2 MB（.pth）	**3.2 MB**（.bin）	压缩约 68%，满足 4 GB 板卡部署

一句话结论：本作品做到了"**压缩 68% 体积，精度基本不掉**"—— 这既是算法设计为量化预留空间的结果，也是真实数据校准策略的直接收益。

检测效果可视化

左起：前时相 → 后时相 → 人工标注真值 → 模型预测

典型正确案例：建筑区整体变化

T1 前时相	T2 后时相	GT 真值	Pred 预测

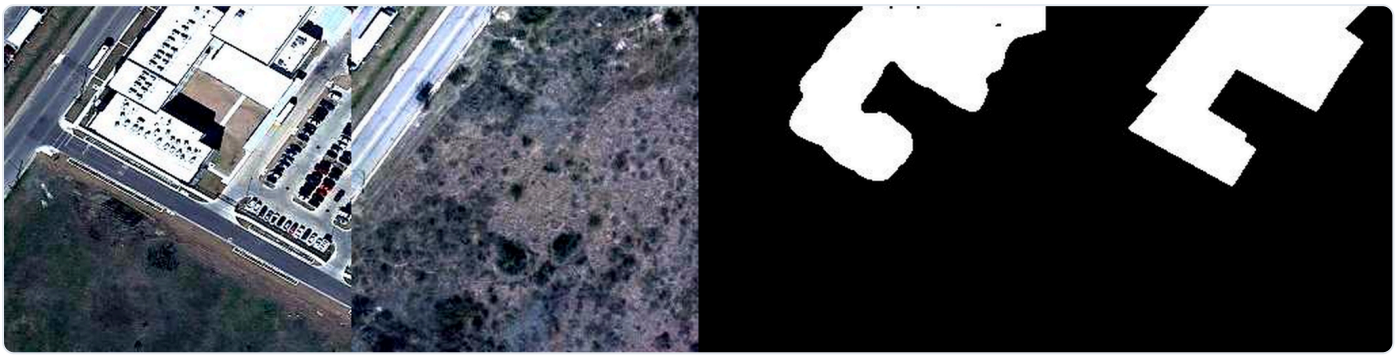


图 4　前时相为密集建筑与绿地，后时相建筑已拆除变为裸地；模型将整片变化区域完整检出，边界与真值高度吻合（预测变化占比 0.1783 vs 真值 0.1897）

典型误报案例：小目标误判

T1 前时相	T2 后时相	GT 真值	Pred 预测



图 5　真值为无变化区域，模型在若干位置产生了零散的小块误报（预测 0.0263 vs 真值 0）。误报面积小、分布零散，可通过置信度阈值与后处理连通域过滤进一步抑制

误差分析结论：模型的主要误差来自**细小块状目标的误报**，而非大面积漏检。 这类误差对变化面积的统计影响有限，且可通过阈值调节与形态学后处理有效抑制—— 这为后续在板端加入**自适应阈值**与**连通域过滤**提供了明确方向。

跑在板子上的完整系统

不是一段推理脚本，而是可操作、可演示、可交付的工程化工作站

能力	说明
变化检测	选择影像目录与 A/B 影像对，一键推理，输出预测掩膜、概率热力图与叠加图
精度评估	对照人工标注，实时计算 Precision / Recall / F1 / IoU 与混淆矩阵（TP / FP / FN / TN）
误差分析	四色误差图直观区分：**TP 正确变化**、**FP 误报**、**FN 漏检**、**TN 正确负例**，并支持点选变化区域高亮定位
AI 语义解释	接入视觉大模型，自动用自然语言描述"哪里变了、变成了什么"，把像素结果翻译成业务语言
设备监控	板端温度、内存、CPU 占用实时折线图，体现边缘设备的真实运行状态
一键导出	把当前结果图、指标、AI 文字一次性导出，便于归档与汇报

端侧工程细节

- **界面自适应**：针对板载 1024×600 屏做了布局适配，可直接用触摸操作；
- **开机即用**：提供桌面快捷方式，一键启动服务并自动全屏打开工作页面；
- **无需外网**：检测与评估全链路本地完成，仅在调用 AI 解释时才需要联网。

> **为什么这很重要：**在 4 GB 内存的板卡上同时跑推理服务、图像处理与前端渲染， 本身就是一次真实的资源约束下的工程权衡。系统能稳定运行，说明端侧承载完整业务闭环是可行的。

答辩要点与话术

针对 AI + 硬件赛道，评委最可能追问的三件事

三个必须讲清楚的点

评委关心	建议话术
为什么算法要这么设计	"我们的网络在**设计阶段就为量化做了预留**——全卷积结构、权重共享、没有动态控制流，所以转 int8 时不会遇到算子不支持或数值坍塌。如果换成一个带注意力的重网络，量化环节很可能直接卡住。"
量化为什么没掉精度	"关键在**校准数据是真实的。**我们踩过坑：一开始用 `skip` 模式让工具随机校准，板端输出全 0。后来改成从训练集抽 60 组真实影像、按训练同款预处理生成校准集，量化后才恢复到 F1 0.8648，和浮点模型基本持平。"
这套方案能复制吗	"能。链路是通用的：**换成别的网络，只要改 march 和输入规格两个参数就能复用。**我们把它整理成了完整教程，包括六类常见报错的定位与解决。"

预判提问与应对

为什么不用大模型	端侧没有 GPU，4 GB 内存装不下；本任务需要的是**像素级定位**而非语义描述，轻量专用网络在精度与成本上更优。实际上我们还把大模型用在了"解释结果"这一环，各司其职。
精度还能再高吗	可以，但当前瓶颈不在算法而在**量化策略。**后续计划引入**混合精度量化**（敏感层保 fp16）与**自适应阈值**，预期能在体积增加可控的前提下进一步提点。
实时性怎么样	单帧 256×256 双时相输入，在 BPU 上以 latency 模式编译、单核运行。**具体帧率与功耗数据尚未系统测量**，这是我们下一步要补齐的测试项。
数据从哪来	训练与评测使用公开数据集 LEVIR-CD（637 对 1024×1024 影像，裁剪为 256×256 后共 10192 组），另在 UAV 数据集上验证了跨数据集泛化能力。

> **诚实建议：**上面"实时性与功耗"那一项目前确实缺数据。 建议在赛前补一次实测——评委问到吞吐、延迟或能效时，有数字比有说法更可靠。

一句话收尾

> 我们做的不是"把模型搬到板子上"，而是**让算法与硬件互相妥协、共同设计**—— 在 4 GB 内存的国产边缘算力上，把像素级遥感解译的精度、体积与可运行性同时做到了可交付的水平。